

LAB GUIDELINES

Lab 10 — Server Roles: Web, File, and Database

Maps to CompTIA Server+ objective **2.3** — Server roles requirements: Print, Database, File, Web, Application, Messaging. This lab builds the three most commonly tested roles end-to-end.

Run all commands on <https://killercode.com/playgrounds/scenario/ubuntu>

Required software (free, apt): nginx, samba, mariadb-server, curl, cifs-utils

```
apt update && apt install -y nginx samba mariadb-server curl cifs-utils
```

Step 1 — Web server (nginx)

```
systemctl start nginx
echo "<h1>Server+ Web Role</h1>" > /var/www/html/index.html
curl -s http://127.0.0.1/ | head
```

Confirm role requirements: TCP/80, TCP/443, document root permissions, log location `/var/log/nginx/`.

Add a virtual host (multi-site web role):

```
mkdir -p /var/www/site1
echo "<h1>site1</h1>" > /var/www/site1/index.html
cat > /etc/nginx/sites-available/site1 <<'EOF'
server {
    listen 80;
    server_name site1.lab.local;
    root /var/www/site1;
    index index.html;
    access_log /var/log/nginx/site1.access.log;
}
EOF
ln -sf /etc/nginx/sites-available/site1 /etc/nginx/sites-enabled/
nginx -t && systemctl reload nginx
echo "127.0.0.1 site1.lab.local" >> /etc/hosts
curl -s http://site1.lab.local/ | head
```

Step 2 — File server (Samba — CIFS/SMB)

apt install smbclient

```
groupadd -f fileshare
useradd -m -G fileshare alice 2>/dev/null
(echo 'P@ssw0rd!'; echo 'P@ssw0rd!') | smbpasswd -s -a alice

mkdir -p /srv/share && chgrp fileshare /srv/share && chmod 2770 /srv/share
echo "shared data" > /srv/share/readme.txt

cat >> /etc/samba/smb.conf <<'EOF'
[share]
    path = /srv/share
    valid users = @fileshare
    read only = no
    create mask = 0660
    directory mask = 2770
EOF

systemctl restart smbd
smbclient -L //127.0.0.1 -U alice%P@ssw0rd!
smbclient //127.0.0.1/share -U alice%P@ssw0rd! -c 'ls'
```

File role checklist: TCP/445, authentication backend, ACLs/permissions, quotas (see Lab 12 + 17), audit logging.

Step 3 — Database server (MariaDB)

```
systemctl start mariadb
mysql -e "CREATE DATABASE appdb;"
mysql -e "CREATE USER 'app'@'localhost' IDENTIFIED BY 'AppPass!1';"
mysql -e "GRANT ALL ON appdb.* TO 'app'@'localhost'; FLUSH PRIVILEGES;"
mysql -uapp -pAppPass!1 appdb -e "CREATE TABLE t (id INT PRIMARY KEY, v VARCHAR(40)); INSERT INTO t"
mysql -uapp -pAppPass!1 appdb -e "SELECT * FROM t;"
```

Database role checklist: TCP/3306, dedicated service account, **least privilege** GRANTs, separate disk for data + binlogs (capacity vs. utilisation, Lab 12), regular logical + physical backups (Lab 26).

Step 4 — Role isolation on a single host (defence in depth)

For a Server+ deployment, never run web + DB + file on one host without:

1. ****Separate service accounts**** (`www-data`, `mysql`, `smbd` users).
2. ****Firewall rules per port**** (Lab 9) — DB only from web tier, file share only from internal subnet.
3. ****Resource limits**** — `systemctl set-property` CPUQuota / MemoryMax to stop one role starving the others.

```
systemctl set-property mariadb.service MemoryMax=512M CPUQuota=80%
systemctl show mariadb -p MemoryMax,CPUQuota
```

Step 5 — Document the role inventory

```
{
  echo "Host: $(hostname -f)"
  echo "Web:      $(systemctl is-active nginx)    port $(ss -tlnH 'sport = :80' | wc -l)"
  echo "File:      $(systemctl is-active smbd)     port $(ss -tlnH 'sport = :445' | wc -l)"
  echo "Database: $(systemctl is-active mariadb) port $(ss -tlnH 'sport = :3306' | wc -l)"
} | tee /root/role-inventory.txt
```

Feed this into the asset management process (Lab 17).

Free online tools

- ****nginx beginner's guide**** — https://nginx.org/en/docs/beginners_guide.html
- ****Samba HOWTO**** — <https://www.samba.org/samba/docs/>
- ****MariaDB Knowledge Base**** — <https://mariadb.com/kb/en/>

Containerised roles (bonus assets)

The same Web / File / Database roles run as containers. Build them with **Docker Desktop** (<https://www.docker.com/products/docker-desktop/>) using the files in assets/:

- assets/Dockerfile — the web role as an nginx image.
- assets/docker-compose.yml — the full Web + File + Database stack:

```
docker compose up -d      # start all three roles
docker compose ps
docker compose down       # stop
```

What you learned

- Standing up the three most-tested server roles (web, file, DB) on a single host.
- Role-specific port, ACL, service-account, and log-location checklists.
- Resource limits via `systemd` to keep coexisting roles from starving each other.
- A simple role-inventory script you can carry into asset management.